pds

Proportionally Dense Subgraphs - Overview

`pds` is a framework written in Python for computing and showing proportionally dense subgraphs (PDSs) of maximum size. Specifically, cubic and k-regular bipartite graphs can be studied and drawn along with their PDSs. Alternatively, the framework can be used to generate random cubic and k-regular bipartite graphs.

Bazgan et al. defined *"a proportionally dense subgraph (PDS) as an induced subgraph of a graph with the property that each vertex in the PDS is adjacent to proportionally as many vertices in the subgraph as in the graph" (source: https://arxiv.org/abs/1903.06579).*

The project also contains several *use cases* that show specific usages of the framework. Some use cases were used to test computationally conjectures about PDSs in cubic and k-regular bipartite graphs. These use cases are described in the section *Usage and Details* (see below).

Installation

On Debian-based Linux

In a `bash` terminal, type:

```
sudo apt update
sudo apt install python3-pip
git clone https://github.com/d-m-3/pds.git
pip install -r requirements.txt
```

On MacOS

1. Check that Python 3 is installed. In a terminal, type:

```
python3 --version
```

2. Download `pip`. In a terminal, type:

```
curl https://bootstrap.pypa.io/get-pip.py -o get-pip.py
```

3. Install `pip`. In a terminal, type:

```
python3 get-pip.py
```

4. Install the modules `networkx` and `matplotlib` with `pip`. In a terminal, type:

```
pip install networkx
pip install matplotlib
```

5. Get the files of the project. Two options:
   a) With `git clone`. In a terminal, type: `git clone https://github.com/d-m-3/pds.git`
   b) Or click on *Code - Download ZIP* on the project's GitHub page: https://github.com/d-m-3/pds

On Windows

1. To install `pip`, follow the `pip` documentation here: https://pip.pypa.io/en/stable/installation/
2. Install the modules `networkx` and `matplotlib` with `pip`. In a terminal, type:

```
pip install networkx
pip install matplotlib
```

3. Get the files of the project. Two options:
   a) With `git clone`. In a terminal, type: `git clone https://github.com/d-m-3/pds.git`
   b) Or click on *Code - Download ZIP* on the project's GitHub page: https://github.com/d-m-3/pds

Usage and Details

- `pds.py` is the main library for finding PDSs, drawing graphs along with their PDSs, and generating random cubic and k-regular bipartite graphs. It can be imported as a module in your project (see *External Usage* below).
- `pds_tests.py` contains the unit tests for all the functions in `pds.py`, except for the functions that draw and/or save graphs.
- `usecase_draw_1_pds.py`
  Goal: Draw a cubic or k-regular bipartite graph and show one PDS of maximum size.
  Execution and details: It creates a random cubic graph on a given number of vertices. It finds a PDS of maximum size, draws the graph, and colors the vertices of the PDS in red. Alternatively, it can create and draw a random k-regular bipartite graph instead of a cubic graph. In that case, a specific layout for bipartite graphs can be used.
- `usecase_draw_all_pds.py`
  Goal: Draw all the possible PDSs of maximum size for a given cubic or k-regular bipartite graph.
  Execution and details: It creates a random cubic graph on a given number of vertices. It finds all its PDSs of maximum size and draws all the different PDSs on different figures (vertices belonging to a PDS are colored in red). Alternatively, it can create a random k-regular bipartite graph instead of a cubic graph. In that case, a specific layout for bipartite graphs can be used.
- `usecase_exceptions_cubic.py`
  Goal: Searches for cubic graphs on *|V|* vertices, with *|V| > 8*, that have no PDS of maximum size.
  Execution and details: It creates random cubic graphs and tests if they do not have a PDS of maximum size. If such a graph with no PDS of maximum size is found, it is drawn, saved as a .png figure and as an edge list that can be imported later, and the program's execution is stopped. The number of vertices in a graph and the number of created and tested graphs can be defined. If the

boolean "only_nh" is set to "True", only cubic graphs that do not have a Hamiltonian cycle are considered (please notice that it takes much longer).

- `usecase_exceptions_k_regular_bipartite.py`
  Goal: Search for k-regular bipartite graphs on *|V|* vertices that have no PDS of maximum size, with *|V| > 8*.
  Execution and details: It creates random k-regular bipartite graphs and tests if they do not have a PDS of maximum size. If such a graph with no PDS of maximum size is found, it is drawn and the program's execution is stopped. The number of vertices in a graph, the value of *k*, and the number of created and tested graphs can be defined.
- `usecase_every_v_in_pds.py`
  Goal: Test the following conjecture computationally "For any cubic graph *G = (V, E)* with *|V| > 8*, every vertex is part of at least one PDS of maximum size".
  Execution and details: It creates random cubic graphs and tests if, for every graph, every vertex is part of at least one PDS of maximum size. If the program finds a graph in which some vertices are not part of any PDS, the graph is drawn, the vertices not belonging to any PDS are displayed, and the program's execution is stopped. The number of vertices in a graph and the number of created and tested graphs can be defined. If the boolean "only_nh" is set to "True", only cubic graphs that do not have a Hamiltonian cycle are considered (please notice that it takes much longer).
- `usecase_every_v_ds2.py`
  Goal: Test the following conjecture computationally "Every cubic graph *G = (V, E)*, with *|V| > 8*, has at least one PDS *S* of maximum size, where *d_S(u) = 2* for every vertex *u* in *S*". This conjecture was proven to be wrong by the finding of counterexamples.
  Execution and details: It creates random cubic graphs and tests if, for every graph, there exists a PDS of maximum size and *d_s(v) = 2* for every vertex *v*. If there is no such PDS for a graph, the graph and all the PDSs of maximum size are drawn, and the program's execution is stopped. The number of vertices in a graph and the number of created and tested graphs can be defined. If the boolean "only_nh" is set to "True", only cubic graphs that do not have a Hamiltonian cycle are considered (please notice that it takes much longer).
- `usecase_every_v_ds2_ds3.py`
  Goal: Test the following conjecture computationally "Every cubic graph *G = (V, E)*, with *|V| > 8*, has at least one PDS *S* of maximum size, where *d_S(u) = d_S(v) = 3* for at most two vertices *u, v* in *S*".
  Execution and details: It creates random cubic graphs and tests if, for every graph, there exists a PDS of maximum size and *d_s(v) = 2* for every vertex *v*, except for at most `ds3_nb` vertices, where *d_s(v) = 3*. If there is no such PDS for a graph, the graph and all the PDSs of maximum size are drawn, and the program's execution is stopped. The number of vertices in a graph and the number of created and tested graphs can be defined. If the boolean "only_nh" is set to "True", only cubic graphs that do not have a Hamiltonian cycle are considered (please notice that it takes much longer).
- `usecase_create_nham_graph.py`
  Goal: Generate, display and save random non-Hamiltonian cubic graphs on a given number of vertices.
  Execution and details: It creates a random non-Hamiltonian cubic graph on a given number of vertices, saves it in the given folder, with the number of vertices at the end of the filename. It also saves the figure in .png format in the same directory.
- `gex.py`. It contains specific graphs used for unit tests and exceptions of cubic graphs on eight vertices, i.e., cubic graphs that do not have a PDS of maximum size.

External Usage

You can import the `pds` library in your own Python project:

```
import pds
```